A 幽幽子想吃东西

直接计算即可，如果 $n \le b$ 就 $-c$ 。

```
void solve() {
    int a, b, c, n;
    cin >> a >> b >> c >> n;
    cout << a*n - c*(n <= b) << "\n";
}
```

Fence 1

B 米斯蒂娅不想被吃掉

在线地处理每一时刻剩余的食材。记截至前一刻剩余的食材为 res ，当前时刻拿到的食材为 t ，由于食材最多能保存两天，所以下一个时刻剩余的食材全部由当前时刻的食材提供，我们优先消耗剩余的食材，所以在下一个时刻剩余的食材就是 $min(t, res + t - x)$ 。

```
void solve() {
    int n, x, res = 0;
    bool f = true;
    cin >> n >> x;
    for (int i = 0; i < n; ++i) {
        int t;
        cin >> t;
        if (t + res >= x) res = min(res + t - x, t);
        else f = false;
    }
    if (f) cout << "Yes" << "\n";
    else cout << "No" << "\n";
}
```

C 三妖精say subsequence！！！

因为三个字母两两不同，我们不妨设三个字母 a<b<c ，那么选择数就是 $cnt[a] \cdot cnt[b] \cdot cnt[c]$ ，在此基础上，三个元素总共有 $3! = 6$ 中排列，所以答案为

$$6 \times \sum_{a<b<c} cnt[a] \cdot cnt[b] \cdot cnt[c] \pmod{998244353}$$

```
void solve() {
    int n, cnt = 0;
    string s;
    cin >> n >> s;
    vector<int> c(26);
    for (auto v : s) c[v - 'a']++;
    for (auto v : c) cnt += v > 0;
    assert(cnt >= 3);
    Z ans = 0;
    for (int i = 0; i < 26; ++i) {
        for (int j = 0; j < 26; ++j) {
            for (int k = 0; k < 26; ++k) {
                if (i != j && j != k && i != k && c[i] && c[j] && c[k]) ans += c[i] * c[j] * c[k];
            }
        }
    }
    cout << ans * 6 << "\n";
}
```

D 恋恋的01串大冒险

贪心即可，一旦达到 k 就置 1 ，然后统计次数对应的位置。

```
void solve() {
    int n, k;
    string s;
    cin >> n >> k >> s;
    vector<int> cnt(n + 1, 0);
    int c = 0, x = 0;
    cnt[0] = 0;
    for (int i = 0; i < n; ++i) {
        if (s[i] == '1') c = 0;
        else ++c;
        if (c == k) {
            ++x;
            c = 0;
        }
        cnt[i + 1] = x;
    }

    vector<int> ans(n + 1, 0);
    int p = 0;
    for (int i = 0; i <= n; ++i) {
        while (p < n && cnt[p + 1] <= i) ++p;
        ans[i] = p;
    }

    for (int i = 0; i <= n; ++i) {
        cout << ans[i] << " ";
    }
    cout << "\n";
}
```

E the world

把时间展开成状态机然后做 BFS 。

1. 状态：我们需要记录四个维度的信息 $(x, y, \text{parity}, \text{mode})$ 。
   - x, y ： 当前坐标。
   - parity ： 当前时间的奇偶性（ 1 为奇数秒， 0 为偶数秒）。因为幽灵每 2 秒循环一次，只需记录模 2 的值。
   - mode ： 时停技能的状态。
     - 0 ： 技能未使用。
     - 1 ： 技能激活中（第1秒），幽灵位置被冻结。
     - 2 ： 技能已使用完毕。
2. **幽灵位置**：
   - 使用两个二维 vector （ $\text{gst}[1]$ 和 $\text{gst}[0]$ ）分别标记奇数秒和偶数秒的幽灵位置。
3. **状态转移**：
   - 普通移动：时间 parity 翻转，mode 保持 0 或 2。检查目标点在新时间是否有幽灵。
   - 发动时停（仅 $\text{mode} = 0$ 可用）： 时间 parity 翻转，mode 变为 1 。因为幽灵被冻结在发动时刻（当前时间 parity ），所以检查目标点在当前时间是否有幽灵。
   - 时停持续（仅 $\text{mode} = 1$ 可用）： 这是时停的第 2步。时间 parity 翻转， mode 变为 2 。幽灵依然被冻结在发动时刻（即上一步的时间，也就是现在的 1-parity ），所以检查目标点在上一步的时间是否有幽灵。

```cpp
int dx[] = {0, 0, 1, -1}, dy[] = {1, -1, 0, 0};

struct State {
    int x, y, p, m;
};

void solve() {
    int n, m, k;
    cin >> n >> m >> k;

    vector<vector<vector<bool>>> gst(2, vector<vector<bool>>(n + 1, vector<bool>(m + 1)));

    for (int i = 0; i < k; ++i) {
        int x1, y1, x2, y2;
        cin >> x1 >> y1 >> x2 >> y2;
        gst[1][x1][y1] = true;
        gst[0][x2][y2] = true;
    }

    auto encode = [&](int x, int y, int p, int mod) -> int {
        return ((x * (m + 1) + y) * 2 + p) * 3 + mod;
    };

    int N = (n + 1) * (m + 1) * 2 * 3;
    vector<int> dist(N, -1);
```

```

    queue<State> q;

    dist[encode(1, 1, 1, 0)] = 0;
    q.push({1, 1, 1, 0});

    int ans = -1;

    while (!q.empty()) {
        auto cur = q.front();
        q.pop();

        int x = cur.x, y = cur.y, p = cur.p, mode = cur.m;
        int d = dist[encode(x, y, p, mode)];

        if (x == n && y == m) {
            ans = d;
            break;
        }

        for (int i = 0; i < 4; ++i) {
            int nx = x + dx[i];
            int ny = y + dy[i];

            if (nx < 1 || nx > n || ny < 1 || ny > m) continue;

            if (mode != 1) {
                int np = 1 - p;
                int nm = (mode == 2 ? 2 : 0);
                if (!gst[np][nx][ny]) {
                    int idx = encode(nx, ny, np, nm);
                    if (dist[idx] == -1) {
                        dist[idx] = d + 1;
                        q.push({nx, ny, np, nm});
                    }
                }
            }
            if (mode == 0) {
                int np = 1 - p;
                int nm = 1;
                if (!gst[p][nx][ny]) {
                    int idx = encode(nx, ny, np, nm);
                    if (dist[idx] == -1) {
                        dist[idx] = d + 1;
                        q.push({nx, ny, np, nm});
                    }
                }
            }

            if (mode == 1) {
                int np = 1 - p;
                int nm = 2;
                if (!gst[1 - p][nx][ny]) {
                    int idx = encode(nx, ny, np, nm);
                    if (dist[idx] == -1) {
                        dist[idx] = d + 1;
```

```
                        q.push({nx, ny, np, nm});
                    }
                }
            }
        }
    }

    cout << ans << "\n";
}
```

Fence 5

F 永远亭的小游戏（续）

每个整数 a_i 分解质因数，记下每个质因子的幂的奇偶性（只保留奇数次因子）。例如 $12 = 2^2 \cdot 3^1 \to$ 只有质因子 3 的指数为奇数 $\to$ 在对应质数位上置 1 。

把所有可能的质数（因为 $a_i \le 100$ ，质数最多 25 个）固定顺序，每个 a_i 可映射成一个长度 ≤ 25 的二进制掩码 $\mathrm{mask[i]}$ 。

若选择若干元素的乘积为完全平方，等价于这些向量按位相加得到全零向量（每个质因子指数和为偶数）。也就是在向量空间中存在非空子集使得异或为 0 —— 等价于这些向量线性相关（不全部线性无关）。

因此，对区间 $[\mathrm{l},\mathrm{r}]$ 问是否存在非空子集乘积为完全平方，等价于判断区间内向量集合的线性相关性，即：秩 $<$ 区间长度。

由于 M 很小（ ≤ 100 ），每次为至多 M 个向量做高斯消元（每向量 ≤ 25位）是非常快的。

此外，根据鸽笼原理，区间长度超过 25，区间内的向量组必然线性相关，所以可以直接判 Yes

```
vector<int> primes, spf(maxn + 1);
int P;

void init() {
    for (int i = 2; i <= maxn; ++i) {
        if (!spf[i]) primes.push_back(i), spf[i] = i;
        for (int j = 0; primes[j]*i <= maxn; ++j) {
            int m = primes[j] * i;
            spf[m] = primes[j];
            if (i % primes[j] == 0) break;
        }
    }
    P = primes.size();
}

void solve() {
    int n, q;
    cin >> n >> q;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; ++i) cin >> a[i];

    vector<int> mask(n + 1, 0);
    for (int i = 1; i <= n; ++i) {
        int x = a[i], m = 0;
        for (int j = 0; j < P; ++j) {
            int cnt = 0;
            while (x % primes[j] == 0) {
                x /= primes[j];
                cnt ^= 1;
            }
            if (cnt) m |= (1 << j);
        }
        mask[i] = m;
    }

    while (q--) {
```

```
        int l, r;
        cin >> l >> r;
        int len = r - l + 1;

        if (len > P) {
            cout << "Yes" << "\n";
            continue;
        }

        vector<int> masks;
        bool z = false;
        for (int i = l; i <= r; ++i) {
            if (mask[i] == 0) {
                z = true;
                break;
            }
            masks.pb(mask[i]);
        }
        if (z) {
            cout << "Yes" << "\n";
            continue;
        }

        sort(all(masks));
        bool f = false;
        for (int i = 1; i < masks.size(); ++i) {
            if (masks[i] == masks[i - 1]) {
                f = true;
                break;
            }
        }
        if (f) {
            cout << "Yes" << "\n";
            continue;
        }

        int rank = 0;
        vector<int> basis(P, 0);
        for (int m : masks) {
            int cur = m;
            for (int b = P - 1; b >= 0; --b) {
                if ((cur >> b) & 1) {
                    if (!basis[b]) {
                        basis[b] = cur;
                        ++rank;
                        break;
                    } else {
                        cur ^= basis[b];
                    }
                }
            }
        }

        if (rank < len) cout << "Yes" << "\n";
        else cout << "No" << "\n";
    }
```

```
}
```

Fence 6

当然也可以力大砖飞，我们维护一个「带位置的线性基」对前缀处理，然后把所有以 r 为右端点的查询离线到 r 上回答：

- 按位置从左到右处理数组，当前处理到下标 i 时，把 $\mathrm{mask[i]}$ 插入到全局线性基，但基中每个基向量同时记录其“位置 pos ”（即这个基向量来源于哪个下标）。
- 插入规则：
  - 用从高位到低位遍历 bit 。
  - 当发现某一位 b 在当前向量 cur 为 1：
    - 如果基中 $\mathrm{basis[b]}$ 为空，则把 cur 放入 $\mathrm{basis[b]}$ ，并记录 `pos[b] = i` ，插入结束。
    - 否则比较位置：若现有 $pos[b] < i$ ，把基和当前向量交换（保证基中存在的向量尽可能来自更靠右的位置），然后对 `cur ^= basis[b]` 继续消去。
  - 若 cur 最终变为 0，则说明新向量依赖于已有基（不插入）。
- 维护的性质：对当前前缀 1..r ， basis 中保存的是一组「相互独立」的向量，且每个 $\mathrm{basis[b]}$ 的 $\mathrm{pos[b]}$ 是该基向量在原数组中出现的下标（并且是该“主位”对应向量中记录的最大下标）。基能表示前缀中所有向量的线性空间。

回答区间 $\mathrm{[l,r]}$ ：

- 已知要用的是前缀 1..r 的基（我们在处理到 r 时存好了基和 pos ）。
- 区间 $\mathrm{[l,r]}$ 中的向量在基里的秩等于基中位置不小于 l 的基向量数量（只有这些基向量来源于区间内，可以作为区间内独立向量的代表；而所有来自 $< l$ 的基向量对应的独立方向仅由区间外向量贡献，不能用于区间内的独立性计数）。
- 于是区间秩 $rank = count(pos[b] >= l)$ 。如果 $rank < r - l + 1$ 则说明区间内向量数大于秩，存在线性相关的非空子集 $\rightarrow$ 答案 Yes ，否则 No 。

```cpp
vector<int> primes, spf(maxn + 1);
int P;

void init() {
    for (int i = 2; i <= maxn; ++i) {
        if (!spf[i]) primes.push_back(i), spf[i] = i;
        for (int j = 0; primes[j]*i <= maxn; ++j) {
            int m = primes[j] * i;
            spf[m] = primes[j];
            if (i % primes[j] == 0) break;
        }
    }
    P = primes.size();
}

void solve() {
    int n, q;
    cin >> n >> q;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; ++i) cin >> a[i];

    vector<int> mask(n + 1, 0);
    for (int i = 1; i <= n; ++i) {
```

```cpp
        int x = a[i];
        int m = 0;
        for (int i = 0; i < P; ++i) {
            int v = primes[i], cnt = 0;
            while (x % v == 0) {
                x /= v;
                cnt ^= 1;
            }
            if (cnt) m |= (1 << i);
        }
        mask[i] = m;
    }

    vector<vector<PII>> qs(n + 1);
    for (int i = 0; i < q; ++i) {
        int l, r;
        cin >> l >> r;
        qs[r].pb({l, i});
    }

    vector<string> ans(q);
    vector<int> bas(P), pos(P);

    auto insert = [&](int x, int idx) {
        int cur = x;
        int cpos = idx;
        for (int b = P - 1; b >= 0 && cur; --b) {
            if (!((cur >> b) & 1)) continue;
            if (bas[b] == 0) {
                bas[b] = cur;
                pos[b] = cpos;
                return;
            }
            if (pos[b] < cpos) {
                swap(pos[b], cpos);
                swap(bas[b], cur);
            }
            cur ^= bas[b];
        }
    };

    for (int r = 1; r <= n; ++r) {
        insert(mask[r], r);
        for (auto [l, idx] : qs[r]) {
            int cnt = 0;
            for (int b = 0; b < P; ++b) if (pos[b] >= l) ++cnt;
            int total = r - l + 1;
            ans[idx] = (cnt < total) ? "Yes" : "No";
        }
    }

    for (int i = 0; i < q; ++i) cout << ans[i] << "\n";
}
```

头文件

```cpp
//Another
#include<bits/stdc++.h>
#include<bits/extc++.h>
#define pb push_back
#define eb emplace_back
#define fi first
#define se second
#define all(a) a.begin(), a.end()
#define rall(a) a.rbegin(), a.rend()
using namespace std;

typedef long long ll;
typedef long double ld;
typedef unsigned long long ull;
typedef __int128 i128;
typedef pair<int, int> PII;
typedef pair<ll, ll> PLL;
typedef tuple<ll, ll, ll> TLLL;
typedef __gnu_pbds::tree<PLL, __gnu_pbds::null_type, less<PLL>, __gnu_pbds::rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update> Tree;
// typedef __gnu_pbds::tree<ll, __gnu_pbds::null_type, less<ll>, __gnu_pbds::rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update> Tree;

constexpr int inf = (ll)1e9 + 7;
constexpr ll INF = (ll)2e18 + 9;
// constexpr ll INF = (ll)4e18;
// constexpr ll MOD = 1e9 + 7;
constexpr ll MOD = 998244353;
constexpr ld PI = acos(-1.0);
constexpr ld eps = 1e-10;

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
ull randint(ull l, ull r) {uniform_int_distribution<unsigned long long> dist(l, r); return dist(rng);}

void init() {

}

void solve() {

}

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    init();

    int t = 1;
```

```
    cin >> t;
    for (int _ = 1; _ <= t; ++_) {
        solve();
    }

    return 0;
}
```

自动取模

```cpp
template<class T>
constexpr T ksm(T a, ll b) {
    T res = 1;
    while (b) {
        if (b & 1) res *= a;
        b >>= 1;
        a *= a;
    }
    return res;
}

template<int P>
struct MInt {
    int x;
    constexpr MInt(): x() {}
    constexpr MInt(ll x) : x{norm(x % getMod())} {}
    static int Mod;
    constexpr static int getMod() {
        if (P > 0) {
            return P;
        } else {
            return Mod;
        }
    }
    constexpr static void setMod(int Mod_) {
        Mod = Mod_;
    }
    constexpr int norm(int x) const {
        if (x < 0) {
            x += getMod();
        }
        if (x >= getMod()) {
            x -= getMod();
        }
        return x;
    }
    constexpr int val() const {
        return x;
    }
    explicit constexpr operator int() const {
        return x;
    }
    constexpr MInt operator-() const {
        MInt res;
        res.x = norm(getMod() - x);
        return res;
    }
    constexpr MInt inv() const {
        assert(x != 0);
        return ksm(*this, getMod() - 2);
    }
    constexpr MInt &operator*=(MInt rhs) & {
```

```
        x = 1LL * x * rhs.x % getMod();
        return *this;
    }
    constexpr MInt &operator+=(MInt rhs) & {
        x = norm(x + rhs.x);
        return *this;
    }
    constexpr MInt &operator-=(MInt rhs) & {
        x = norm(x - rhs.x);
        return *this;
    }
    constexpr MInt &operator/=(MInt rhs) & {
        return *this *= rhs.inv();
    }
    friend constexpr MInt operator*(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res *= rhs;
        return res;
    }
    friend constexpr MInt operator+(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res += rhs;
        return res;
    }
    friend constexpr MInt operator-(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res -= rhs;
        return res;
    }
    friend constexpr MInt operator/(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res /= rhs;
        return res;
    }
    friend constexpr istream &operator>>(istream &is, MInt &a) {
        ll v;
        is >> v;
        a = MInt(v);
        return is;
    }
    friend constexpr ostream &operator<<(ostream &os, const MInt &a) {
        return os << a.val();
    }
    friend constexpr bool operator==(MInt lhs, MInt rhs) {
        return lhs.val() == rhs.val();
    }
    friend constexpr bool operator!=(MInt lhs, MInt rhs) {
        return lhs.val() != rhs.val();
    }
};

template<>
int MInt<0>::Mod = 998244353;

template<int V, int P>
constexpr MInt<P> CInv = MInt<P>(V).inv();
```

```

using Z = MInt<MOD>;
```

Fence 9